

Apache Paimon

MergeTree 模块

核心架构设计文档

Core Architecture Design Document

项目名称	Apache Paimon
模块名称	paimon-core / mergetree
分支版本	release-1.3.1
文档类型	核心架构设计文档
源码路径	org.apache.paimon.mergetree
文件数量	114 个 Java 源文件
子目录	根目录(13) + compact(35+) + localmerge(3) + lookup(7+)

目 录

1. 模块概述与定位
2. 整体架构设计
3. LSM-Tree 核心数据结构
4. 写入流程架构
5. Compaction 压缩机制
6. 合并读取机制
7. MergeFunction 合并函数体系
8. MergeFunctionWrapper 装饰器体系
9. Lookup 查找优化机制
10. Changelog 变更日志生产
11. LocalMerger 本地预合并
12. 设计模式分析
13. 关键场景调用链
14. 核心类清单与职责

1 模块概述与定位

1.1 模块简介

MergeTree 模块是 Apache Paimon 存储引擎的核心组件，位于 paimon-core 模块中的 `org.apache.paimon.mergetree` 包下。该模块实现了基于 LSM-Tree (Log-Structured Merge Tree) 的高性能存储引擎，负责数据的写入、存储、合并和读取等核心操作。

MergeTree 模块采用分层架构设计，将数据文件组织为多个层级 (Levels)，通过后台 Compaction 机制不断优化存储布局，同时提供可插拔的 MergeFunction 策略来支持不同的数据合并语义。该模块是 Paimon 实现高吞吐写入、高效点查和流式变更捕获的技术基石。

1.2 核心能力

- 高吞吐写入：基于内存 WriteBuffer + 批量 Flush 机制，支持高并发写入场景
- LSM-Tree 存储：多层级文件组织，平衡写放大和读放大
- 可插拔合并策略：支持去重、部分更新、聚合、首行保留等多种 MergeFunction
- 自适应压缩：Universal Compaction 策略，支持大小放大、大小比率、文件数量等触发条件
- 点查优化：LookupLevels 提供基于本地 SST 缓存的高效 Key 查找
- 变更日志：支持 INPUT、FULL_COMPACTION、LOOKUP 三种 Changelog 产生模式
- 本地预合并：HashMapLocalMerger / SortBufferLocalMerger 减少写入放大
- 高效多路归并：LoserTree 算法实现高效的多路有序数据合并

1.3 模块结构

MergeTree 模块包含 114 个 Java 源文件，按功能划分为 4 个目录层级：

目录	核心内容
根目录 (13文件)	MergeTreeWriter、Levels、SortedRun、WriteBuffer、MergeSorter、LookupLevels、MergeTreeReaders、DataFileReader 等
compact/ (35+ 文件)	CompactManager、CompactStrategy、CompactRewriter、MergeFunction、UniversalCompaction、SortMergeReader、LoserTree、IntervalPartition 等
localmerge/ (3文件)	LocalMerger 接口、HashMapLocalMerger、SortBufferLocalMerger
lookup/ (7+ 文件)	PersistProcessor 体系、LookupSerializerFactory、RemoteFileDownloader、FilePosition、PositionedKeyValue

2 整体架构设计

2.1 架构总览

MergeTree 模块采用分层架构设计，从上至下依次为：写入层 (Writer Layer)、存储层 (Storage Layer)、压缩层 (Compaction Layer)、合并层 (Merge Layer) 和查找层 (Lookup Layer)。各层之间通过清晰的接口和数据流进行解耦协作。

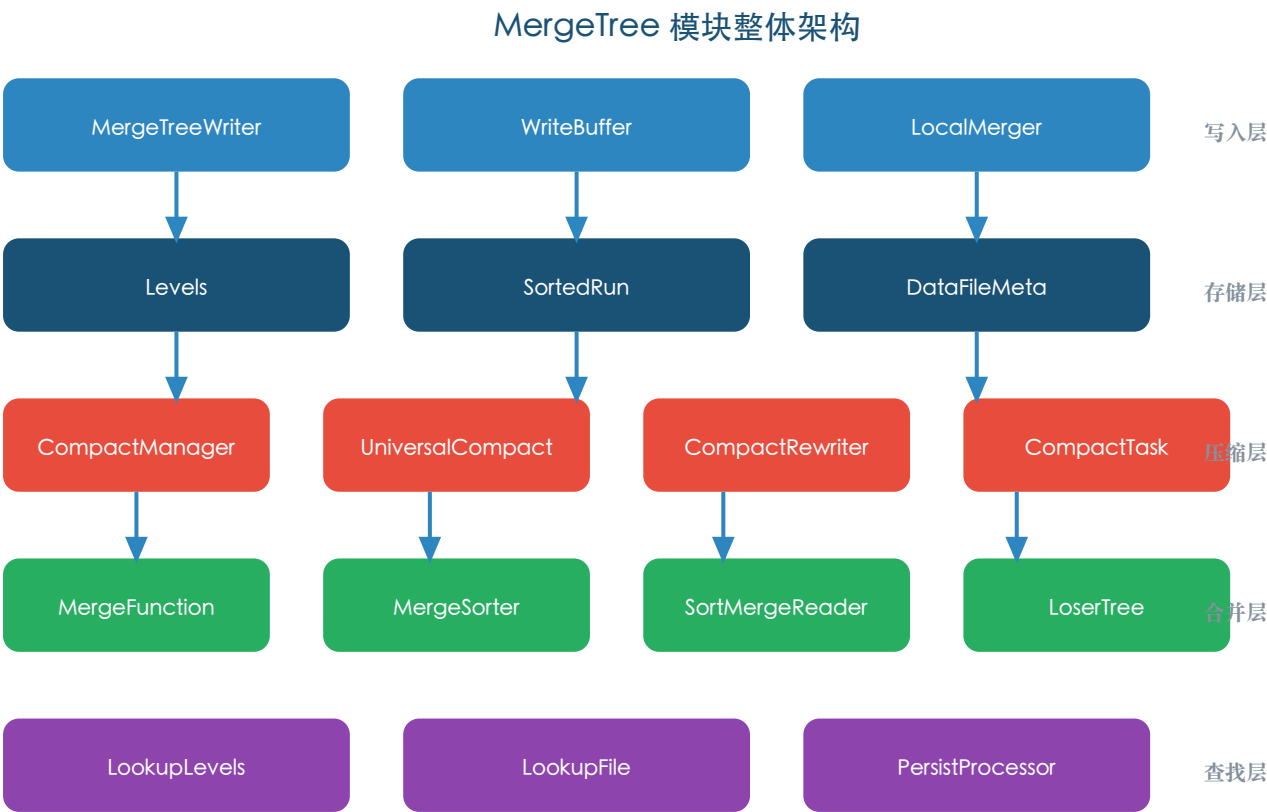


图 2-1 MergeTree 模块整体架构图

2.2 核心数据流

- 写入：MergeTreeWriter.write() → WriteBuffer → Level 0 文件
- 预合并：LocalMerger 在写入前预聚合相同 Key 的记录
- Flush：WriteBuffer 达到阈值 → 排序输出 → 写入 Level 0 SST 文件
- Compaction：CompactManager 根据策略选择文件 → CompactRewriter 重写到高层级
- 读取：MergeTreeReaders 按 Section 组织 → SortMergeReader 多路归并
- 查找：LookupLevels 基于 Key 点查 → 本地 SST 缓存加速

2.3 核心接口体系

接口	职责
----	----

WriteBuffer	写入缓冲区: put/size/memoryOccupancy/flushMemory/forEach/clear
MergeFunction	合并函数: reset/add/getResult, 可插拔合并语义
MergeFunctionFactory	合并函数工厂: 根据 ReadType 投影创建 MergeFunction
MergeFunctionWrapper	合并函数装饰器: 为 MergeFunction 增加额外功能
CompactStrategy	压缩策略: pick() 选择待压缩文件集合
CompactRewriter	压缩重写器: rewrite/upgrade 处理文件重写
LocalMerger	本地预合并: put/size/forEach/clear
PersistProcessor	查找值序列化: Lookup 值的持久化和反序列化

3 LSM-Tree 核心数据结构

3.1 Levels 层级管理

Levels 类是 MergeTree 存储层的核心，管理所有层级的数据文件。Level 0 使用 TreeSet<DataFileMeta> 按序列号排序，每个文件独立作为一个 SortedRun；Level 1+ 使用 List<SortedRun>，每个 SortedRun 内文件按 Key 范围有序且不重叠。

Level 0 特殊处理：TreeSet 按文件序列号排序，每个文件独立为一个 SortedRun

高层级管理：List<SortedRun> 管理 Level 1+，文件按 Key 范围有序不重叠

DropFileCallback：文件删除回调，通知 LookupLevels 缓存失效

快照能力：allFiles() 和 levelSortedRuns() 提供当前文件布局的不可变快照

3.2 SortedRun 有序文件集

SortedRun 表示一组按 Key 排序的不重叠数据文件列表，是不可变值对象。提供 empty()、fromSingle()、fromSorted()、fromUnsorted() 工厂方法。通过 nonOverlapping() 验证文件间无键范围重叠。

3.3 IntervalPartition 区间分区算法

IntervalPartition 基于优先队列将文件按键区间重叠关系划分为最少数量的 SortedRun。用于 Compaction 过程中组织待压缩文件集合。算法：将文件按 minKey 排序，用 PriorityQueue 追踪每个 SortedRun 的 maxKey，贪心分配到不重叠的最佳 SortedRun。

4 写入流程架构

4.1 MergeTreeWriter 核心写入器

MergeTreeWriter 实现 RecordWriter 和 MemoryOwner 接口，管理写入缓冲区、Flush、Compaction 触发和提交增量追踪。关键字段包括 WriteBuffer、CompactManager、Levels、userKeyComparator 和单调递增的 newSequenceNumber。

4.2 SortBufferWriteBuffer

SortBufferWriteBuffer 基于 BinaryInMemorySortBuffer 实现内存排序缓冲，支持可选的 BinaryExternalSortBuffer 磁盘溢写。内置 MergeIterator 在遍历时对相同 Key 执行 MergeFunction 合并，实现排序与合并的流水线处理。

4.3 写入时序流程

```
Client -->> MergeTreeWriter.write(record)
|-- [■■■] LocalMerger.put(record) ■■■■
|-- WriteBuffer.put(sequenceNumber, valueKind, key, value)
|-- ■■■ WriteBuffer ■■■■■■
| |-- ■: flushWriteBuffer()
| | |-- SortBufferWriteBuffer.forEach() ■■■■■■
| | |-- FileWriter ■■■■ Level-0 ■■
| | |-- Levels.addLevel0File(newFile)
| | |-- CompactManager.triggerCompaction()
| |-- ■: ■■■■■■
```

4.4 MergeSorter

MergeSorter 支持 MIN_HEAP 和 LOSER_TREE 两种排序引擎，当数据量超过内存限制时自动切换到磁盘溢写模式。是构建排序合并管道的核心组件。

5 Compaction 压缩机制

5.1 CompactManager 压缩管理器

`MergeTreeCompactManager` 继承 `CompactFutureManager`，管理 `Compaction` 生命周期。`triggerCompaction()` 判断是否需要压缩，`submitCompaction()` 提交异步任务。完成后通过 `Levels.update()` 原子更新层级结构。

5.2 UniversalCompaction 通用压缩策略

借鉴 RocksDB 的通用压缩策略，包含三种决策模式：

策略类型	执行逻辑
大小放大压缩	所有文件总大小与最大 Run 比率超过 <code>maxSizeAmp</code> 阈值时，触发全量压缩
大小比率压缩	从最旧 Run 开始，候选集与下一个 Run 的比率超过 <code>sizeRatio</code> 时触发部分压缩
文件数量压缩	<code>SortedRun</code> 总数超过 <code>numRunCompactionTrigger</code> 且无其他条件满足时，选择最旧 Run 合并

5.3 CompactRewriter 重写器体系

```
CompactRewriter (■■■)
■■■■ AbstractCompactRewriter
■■■■ MergeTreeCompactRewriter (■■■■■)
■■■■ ChangelogMergeTreeRewriter (■■■)
■■■■ FullChangelogMergeTreeCompactRewriter
■■■■ LookupMergeTreeCompactRewriter (■■■DeletionVector)
```

5.4 MergeTreeCompactTask

支持分区压缩和大文件升级优化：单 `Section` 且目标 `Level` 相同时执行 `upgrade()`（仅改元数据），否则执行 `rewrite()`（读取合并重写）。

5.5 ForceUpLevel0 与 EarlyFullCompaction

`ForceUpLevel0Compaction` 装饰 `UniversalCompaction`，强制 `Level-0` 文件与高层级合并。`EarlyFullCompaction` 根据时间间隔/总大小/增量大小三个维度触发提前全量压缩。

6 合并读取机制

6.1 MergeTreeReaders 读取器工厂

MergeTreeReaders 将 Levels 中的文件按 Section 组织（键范围重叠的 SortedRun 组），每个 Section 内通过 SortMergeReader 多路归并，不同 Section 间通过 ConcatRecordReader 顺序拼接。

```
MergeTreeReaders.readerForMergeTree()  
|-- sections: List<List<SortedRun>>  
| |-- ■■ Section: MergeSorter.mergeSort() ■■■■  
| | |-- ■■ SortedRun: ConcatRecordReader ■■■■■■  
|-- ConcatRecordReader ■■■■ Section
```

6.2 SortMergeReader 两种实现

实现类	设计特点
SortMergeReaderWithMinHeap	基于 PriorityQueue 的经典实现。每个 RecordReader 用 Element 包装，堆顶始终为最小 Key。逐一弹出相同 Key 的 Element 调用 MergeFunction
SortMergeReaderWithLoserTree	基于败者树的高效实现。adjustAll() 一次产出相同 Key 的批次，减少比较次数，多路归并路数多时性能优势明显

6.3 LoserTree 败者树算法

LoserTree 支持重复 Key 处理，定义 6 种节点状态 (WINNER/LOSER × NEW_KEY/SAME_KEY/POPPED)。adjustAll() 遍历所有内部节点，收集 SAME_KEY 状态记录为一批输出，NEW_KEY 保留为下一批起始，避免逐条弹出开销。

6.4 ConcatRecordReader

将多个不重叠的 RecordReader 按顺序拼接。当前 Reader 耗尽后自动切换下一个。用于 SortedRun 内文件顺序读取和 Section 间拼接。

7 MergeFunction 合并函数体系

7.1 接口设计

`MergeFunction` 定义合并语义：`reset()` 重置 → `add(KeyValue)` 添加 → `getResult()` 获取结果。`requireCopy()` 控制是否需要拷贝 `KeyValue`。

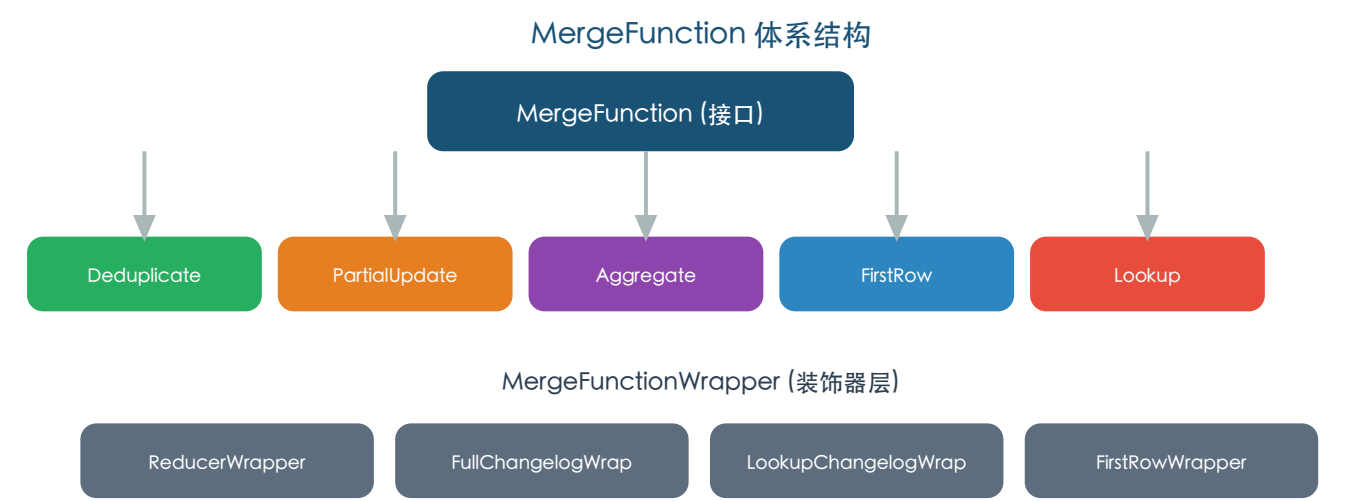


图 7-1 MergeFunction 体系结构图

7.2 五种核心实现

实现类	合并语义
DeduplicateMergeFunction	去重：仅保留最新记录。支持 ignoreDelete 选项
PartialUpdateMergeFunction	部分更新：用新记录非 null 字段更新旧记录。 支持序列组、字段聚合器、remove-on-delete
AggregateMergeFunction	聚合：对每个字段应用 FieldAggregator (sum/min/max/collect 等)
FirstRowMergeFunction	首行保留：保留最早记录。containsHighLevel 判断是否已存在
LookupMergeFunction	Lookup 感知：仅考虑最新高层级记录 + Level-0 增量

8 MergeFunctionWrapper 装饰器体系

8.1 装饰器模式应用

MergeTree 大量使用装饰器模式扩展 MergeFunction 行为，实现合并逻辑与附加功能（Changelog 产生、优化跳过）的解耦。

装饰器	功能
ReducerMergeFunctionWrapper	单条输入时跳过 MergeFunction 调用直接返回，减少合并开销
FullChangelogMergeFunctionWrapper	比较 top-level 与合并结果差异，产生 BEFORE/AFTER 变更记录
LookupChangelogMergeFunctionWrapper	通过 LookupLevels 查找历史值，为 Level-0 记录产生 Changelog
FirstRowMergeFunctionWrapper	结合 Lookup 判断 Key 是否已存在，实现高效首行去重

8.2 典型组合示例

```
// ■■■1: FULL_COMPACTIION changelog
wrapper = new ReducerWrapper(new FullChangelogWrapper(base));

// ■■■2: Lookup ■■
wrapper = new LookupChangelogWrapper(base, lookupLevels, deletionVectors);

// ■■■3: FirstRow ■■
wrapper = new FirstRowWrapper(base, lookupLevels);
```

9 Lookup 查找优化机制

9.1 LookupLevels 层级查找

LookupLevels 基于 Caffeine 缓存构建本地 SST 文件缓存，提供 Key 点查能力。查找流程：LookupUtils 从高层级到低层级 → 二分查找 SortedRun → 匹配 DataFileMeta → 检查 Caffeine 缓存 → 未命中则创建 LookupFile → 支持远程文件下载和 Bloom Filter 优化。

9.2 PersistProcessor 体系

处理器	用途
PersistEmptyProcessor	仅关注 Key 是否存在，不存储值。用于 FirstRow 存在性检查
PersistValueProcessor	序列化完整 Value 内容。用于标准 Lookup 查找
PersistPositionProcessor	存储 FilePosition (文件名+行号)。用于 DeletionVector 场景
PersistValueAndPosProcessor	同时存储 Value 和 FilePosition。用于复合查找场景

10 Changelog 变更日志生产

10.1 三种 Changelog 模式

模式	实现机制
INPUT	从 WriteBuffer 直接产生。每条写入记录都作为 Changelog 输出
FULL_COMPACTION	全量压缩时通过 FullChangelogMergeFunctionWrapper 比较合并前后差异，生成精确 BEFORE/AFTER 记录
LOOKUP	Lookup 压缩时通过 LookupChangelogMergeFunctionWrapper 查找历史值，为 Level-0 增量生成变更日志。支持 DeletionVector

10.2 ChangelogMergeTreeRewriter

抽象基类定义 UpgradeStrategy 枚举 (NO_CHANGELOG / CHANGELOG_WITH_REWRITE / CHANGELOG_NO_REWRITE) 控制文件升级时的 Changelog 行为。子类覆写 createMergeFunctionWrapper() 和 upgradeChangelog() 实现具体逻辑。

10.3 DeletionVector 集成

LOOKUP 模式下 LookupMergeTreeCompactRewriter 支持 DeletionVector：标记删除位置而非重写文件，大幅减少写放大。数据流：PositionedKeyValue → PersistPositionProcessor → LookupFile → DeletionVector。

11 LocalMerger 本地预合并

LocalMerger 在写入 WriteBuffer 前预合并相同 Key 记录，减少写入放大和 Compaction 压力。

实现类	底层结构	合并时机	适用场景
HashMapLocalMerger	BytesHashMap	put() 时原地合并 (FieldSetters)	Key 重复率高 更新密集
SortBufferLocalMerger	SortBufferWriteBuffer	forEach() 时排序合并 (MergeFunction)	Key 分布分散 需排序输出

12 设计模式分析

12.1 设计模式总览

模式	应用位置	具体实现
策略模式	MergeFunction 体系	Deduplicate/PartialUpdate/Aggregate/FirstRow/Lookup 五种可切换策略
策略模式	CompactStrategy	UniversalCompaction/ForceUpLevel0/EarlyFullCompaction 可互换
装饰器模式	MergeFunctionWrapper	Reducer/FullChangelog/LookupChangelog/FirstRow 层层装饰
装饰器模式	CompactStrategy	ForceUpLevel0 和 EarlyFull 装饰 UniversalCompaction
工厂方法	MergeFunctionFactory KvCompactionManagerFactory	工厂接口隐藏复杂创建逻辑
模板方法	CompactRewriter 继承体系	Abstract 定义框架，子类覆写模板方法
观察者模式	Levels.DropFileCallback	文件删除回调通知 LookupLevels 缓存失效
组合模式	ConcatRecordReader	多个 Reader 组合为统一接口
享元模式	LookupFile 缓存	Caffeine 缓存共享 LookupFile 实例

12.2 SOLID 原则

原则	体现
单一职责 (SRP)	Levels 管理层级、SortedRun 管理文件集、MergeFunction 负责合并
开闭原则 (OCP)	MergeFunction/CompactStrategy 接口允许扩展新策略
里氏替换 (LSP)	所有 MergeFunction/CompactRewriter 实现可互换
接口隔离 (ISP)	WriteBuffer/MergeFunction/PersistProcessor 定义最小方法集
依赖倒转 (DIP)	Writer 依赖 WriteBuffer 接口而非具体实现

13 关键场景调用链

13.1 数据写入完整调用链

```
MergeTreeWriter.write(record)
|-- sequenceNumber = ++newSequenceNumber
|-- [LocalMerger] localMerger.put(key, value)
| |-- HashMapLocalMerger: bytesHashMap.lookup(key)
| | |-- ■■■■: fieldSetters.set() ■■■■
| | |-- ■■■■: bytesHashMap.append(key, value)
|-- writeBuffer.put(seq, valueKind, key, value)
|-- if writeBuffer.memoryOccupancy() >= threshold:
| |-- flushWriteBuffer()
| |-- writeBuffer.forEach(merger) ■■■■
| |-- dataFileWriter.write() -> Level-0 SST
| |-- levels.addLevel0File(fileMeta)
| |-- compactManager.triggerCompaction()
```

13.2 Compaction 完整调用链

```
MergeTreeCompactManager.triggerCompaction()
|-- compactStrategy.pick(numLevels, runs)
| |-- UniversalCompaction:
| | 1) pickForSizeAmplification()
| | 2) pickForSizeRatio()
| | 3) pickForFileCount()
|-- ■■ MergeTreeCompactTask
| |-- IntervalPartition.partition(files)
| |-- compactRewriter.rewrite(level, sections)
| | |-- MergeTreeReaders -> SortMergeReader
| | |-- MergeFunction.add(kv) ■■
| | |-- dataFileWriter.write(merged)
|-- CompactFuture.join()
|-- levels.update(before, after)
```

13.3 Lookup 查找调用链

```
LookupLevels.lookup(key, startLevel)
|-- for level in [maxLevel..startLevel]:
| |-- LookupUtils.lookup(key, level, sortedRuns)
| | |-- Level 1+: ■■■■ SortedRun ■■■■
| | |-- Level 0: ■■■■■■■■■■
| |-- ■■■■■■ -> ■■ Caffeine ■■
| | |-- ■■: lookupFile.get(key)
| | |-- ■■■■: createLookupFile(dataMeta)
| | |-- [■■■■■] remoteDownloader.download()
| | |-- ■■ LookupStoreReader
| | |-- cache.put(fileName, lookupFile)
| |-- PersistProcessor.deserialize(bytes)
```


14 核心类清单与职责

14.1 根目录核心类

类名	职责
MergeTreeWriter	核心写入器，管理 WriteBuffer/Compaction/Levels
Levels	层级管理，Level 0 TreeSet + Level 1+ SortedRun
SortedRun	不可变有序文件集，确保文件间无键范围重叠
SortBufferWriteBuffer	内存排序缓冲区，支持磁盘溢写
MergeSorter	多路归并排序器，MIN_HEAP/LOSER_TREE 双引擎
MergeTreeReaders	读取器工厂，构建 Section -> Run -> File 读取管道
LookupLevels	Key 点查，Caffeine 缓存本地 SST 文件
LookupFile	缓存的本地查找文件，Caffeine RemovalListener 管理生命周期
DataFileReader	数据文件读取器，LookupStoreReader 包装
DropDeleteReader	过滤非 ADD RowKind 记录的装饰器 Reader

14.2 compact/ 核心类

类名	职责
MergeTreeCompactManager	Compaction 生命周期管理，触发/等待/更新
UniversalCompaction	通用压缩策略：大小放大/比率/文件数量三种模式
MergeTreeCompactTask	压缩任务执行，支持 upgrade 和 rewrite 两种路径
IntervalPartition	基于优先队列的文件区间分区算法
SortMergeReaderWithMinHeap	最小堆多路归并读取
SortMergeReaderWithLoserTree	败者树多路归并读取
LoserTree	6 状态败者树，高效多路归并+重复 Key 批处理
DeduplicateMergeFunction	去重合并：保留最新记录
PartialUpdateMergeFunction	部分更新合并：非 null 字段更新
AggregateMergeFunction	聚合合并：字段级 FieldAggregator
LookupMergeTreeCompactRewriter	Lookup 压缩重写器，支持 DeletionVector
FullChangelogMergeTreeCompactRewriter	增量压缩 Changelog 重写器

14.3 localmerge/ 与 lookup/ 核心类

类名	职责
----	----

LocalMerger	本地预合并接口
HashMapLocalMerger	BytesHashMap 原地合并
SortBufferLocalMerger	排序缓冲合并
PersistProcessor	Lookup 值序列化/反序列化
PersistEmptyProcessor	空值处理器（存在性检查）
PersistValueProcessor	完整值处理器
PersistPositionProcessor	文件位置处理器（DeletionVector）
RemoteFileDownloader	远程文件下载接口
FilePosition	文件名+行号定位
PositionedKeyValue	带位置信息的 KeyValue